

Conteúdo 12:

Strings

Professora: **Me. Alana Neo**

Slides baseados no material do Prof. Me. Douglas Thames
de Araujo

Strings

- **Strings são objetos/instâncias** da classe `java.lang.String`.
- **String não é um tipo primitivo** do Java, como `int`, `double` ou `boolean`.
- **Elas contém sequências de caracteres** e são utilizadas para armazenar textos
- Strings são bastante utilizadas no Java, por isso estudaremos alguns métodos importantes desta classe

Strings

- Uma String pode ser inicializada com a palavra-chave 'new':

```
String senha = new String("12345");
```

- Ou pode ser inicializada diretamente:

```
String senha = "12345";
```

- Além de poder ser declarada e ter um valor inserido posteriormente:

```
String senha;  
senha = "12345";
```

Strings

- Strings podem ser concatenadas com o operador '+':

```
String nome = "Diego";  
nome = nome + " Oliveira";  
System.out.println(nome);
```

Output - P00-Exercicio14 (run)

```
run:  
Diego Oliveira  
BUILD SUCCESSFUL (total time: 0 seconds)
```

- Ou com o método concat:

```
String nome1 = "Diego";  
String nome2 = "Oliveira";  
System.out.println(nome1.concat(nome2));
```

Output - P00-Exercicio14 (run)

```
run:  
DiegoOliveira
```

Strings

- Vejamos alguns métodos interessantes da classe String:
 - Remover espaços antes e depois de um texto:

```
String nome = "    Diego Oliveira    ";  
System.out.println(nome.trim());
```

- Substituir partes do texto:

```
String ifrn = "IFRN";  
ifrn = ifrn.replace("IF", "Instituto Federal");  
ifrn = ifrn.replace("RN", " Rio Grande do Norte");  
System.out.println(ifrn);
```

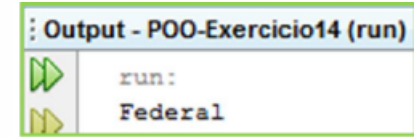
```
Output - POO-Exercicio14 (run)  
run:  
Instituto Federal Rio Grande do Norte
```

Strings

Vejamos alguns métodos interessantes da classe String:

– Recortar uma parte de uma String:

```
String ifrn = "Instituto Federal do Rio Grande do Norte";  
System.out.println(ifrn.substring(10, 17));
```



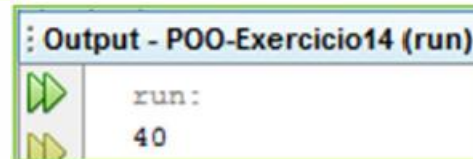
Output - POO-Exercicio14 (run)

run:
Federal

An arrow points from the `substring(10, 17)` method call in the code block above to this output window.

– Tamanho de uma String:

```
String ifrn = "Instituto Federal do Rio Grande do Norte";  
System.out.println(ifrn.length());
```



Output - POO-Exercicio14 (run)

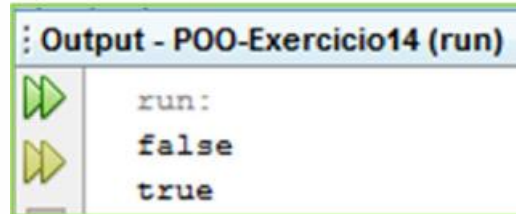
run:
40

Strings

Vejamos alguns métodos interessantes da classe String:

– Comparando Strings:

```
String nome1 = "Diego Oliveira";  
String nome2 = "diego oliveira";  
System.out.println(nome1.equals(nome2));  
System.out.println(nome1.equalsIgnoreCase(nome2));
```



Strings

Vejamos alguns métodos interessantes da classe String:

– Dividindo Strings em partes:

```
String ifrn = "Instituto Federal do Rio Grande do Norte";  
String partes[] = ifrn.split(" ");  
for(int i=0; i<partes.length; i++){  
    System.out.println(partes[i]);  
}
```

Output - POO-Exercicio14 (run)



run:



Instituto

Federal



do



Rio

Grande

do

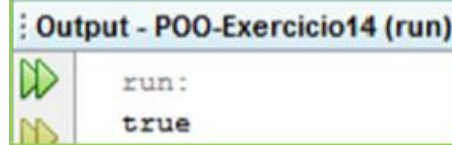
Norte

Strings

Vejamos alguns métodos interessantes da classe String:

– Verificando se a String contém uma palavra:

```
String texto = "Instituto Federal do Rio Grande Do Norte";  
System.out.println(texto.contains("Federal"));
```

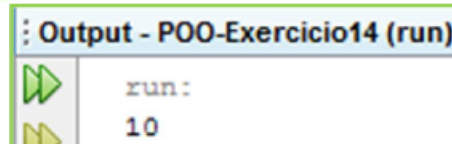


Output - P00-Exercicio14 (run)

run:
true

– Verificando a posição de uma palavra no texto:

```
String texto = "Instituto Federal do Rio Grande Do Norte";  
System.out.println(texto.indexOf("Federal"));
```



Output - P00-Exercicio14 (run)

run:
10

Strings

Vejamos alguns métodos interessantes da classe String:

– Verificando início e fim de uma String:

```
String texto = "Instituto Federal do Rio Grande Do Norte";  
System.out.println(texto.startsWith("Inst"));  
System.out.println(texto.endsWith("rte"));
```

Output - POO-Exercicio14 (run)

```
run:  
true  
true
```

– Pegando o último caractere:

```
String texto = "Instituto Federal do Rio Grande Do Norte";  
System.out.println(texto.charAt(texto.length()-1));
```

Output - POO-Exercicio14 (run)

```
run:  
e
```

Strings

- Vejamos alguns métodos interessantes da classe String:
 - Passando um texto para maiúsculo e minúsculo:

```
String texto = "Instituto Federal do Rio Grande Do Norte";  
System.out.println(texto.toUpperCase());  
System.out.println(texto.toLowerCase());
```

```
Output - POO-Exercicio14 (run)  
run:  
INSTITUTO FEDERAL DO RIO GRANDE DO NORTE  
instituto federal do rio grande do norte
```

Strings

- Cada caractere é representado por um número inteiro no Java. Vejamos a impressão de todas as letras minúsculas e maiúsculas:

```
System.out.println("Caracteres alfabéticos maiúsculos:");  
for (int i = 65; i <= 90; i++) {  
    System.out.printf("%c", i);  
}  
System.out.println("\nCaracteres alfabéticos minúsculos:");  
for (int i = 97; i <= 122; i++) {  
    System.out.printf("%c", i);  
}
```

Output - POO-Exercicio14 (run)



run:



Caracteres alfabéticos maiúsculos:
ABCDEFGHIJKLMNOPQRSTUVWXYZ



Caracteres alfabéticos minúsculos:
abcdefghijklmnopqrstuvwxyzAAA



Strings – Tabela ASCII

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]	48	30	110000	60	0	96	60	1100000	140	`
1	1	1	1	[START OF HEADING]	49	31	110001	61	1	97	61	1100001	141	a
2	2	10	2	[START OF TEXT]	50	32	110010	62	2	98	62	1100010	142	b
3	3	11	3	[END OF TEXT]	51	33	110011	63	3	99	63	1100011	143	c
4	4	100	4	[END OF TRANSMISSION]	52	34	110100	64	4	100	64	1100100	144	d
5	5	101	5	[ENQUIRY]	53	35	110101	65	5	101	65	1100101	145	e
6	6	110	6	[ACKNOWLEDGE]	54	36	110110	66	6	102	66	1100110	146	f
7	7	111	7	[BELL]	55	37	110111	67	7	103	67	1100111	147	g
8	8	1000	10	[BACKSPACE]	56	38	111000	70	8	104	68	1101000	150	h
9	9	1001	11	[HORIZONTAL TAB]	57	39	111001	71	9	105	69	1101001	151	i
10	A	1010	12	[LINE FEED]	58	3A	111010	72	:	106	6A	1101010	152	j
11	B	1011	13	[VERTICAL TAB]	59	3B	111011	73	;	107	6B	1101011	153	k
12	C	1100	14	[FORM FEED]	60	3C	111100	74	<	108	6C	1101100	154	l
13	D	1101	15	[CARRIAGE RETURN]	61	3D	111101	75	=	109	6D	1101101	155	m
14	E	1110	16	[SHIFT OUT]	62	3E	111110	76	>	110	6E	1101110	156	n
15	F	1111	17	[SHIFT IN]	63	3F	111111	77	?	111	6F	1101111	157	o
16	10	10000	20	[DATA LINK ESCAPE]	64	40	1000000	100	@	112	70	1110000	160	p
17	11	10001	21	[DEVICE CONTROL 1]	65	41	1000001	101	A	113	71	1110001	161	q
18	12	10010	22	[DEVICE CONTROL 2]	66	42	1000010	102	B	114	72	1110010	162	r
19	13	10011	23	[DEVICE CONTROL 3]	67	43	1000011	103	C	115	73	1110011	163	s
20	14	10100	24	[DEVICE CONTROL 4]	68	44	1000100	104	D	116	74	1110100	164	t
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]	69	45	1000101	105	E	117	75	1110101	165	u
22	16	10110	26	[SYNCHRONOUS IDLE]	70	46	1000110	106	F	118	76	1110110	166	v
23	17	10111	27	[ENG OF TRANS. BLOCK]	71	47	1000111	107	G	119	77	1110111	167	w
24	18	11000	30	[CANCEL]	72	48	1001000	110	H	120	78	1110000	170	x
25	19	11001	31	[END OF MEDIUM]	73	49	1001001	111	I	121	79	1110001	171	y
26	1A	11010	32	[SUBSTITUTE]	74	4A	1001010	112	J	122	7A	1110100	172	z
27	1B	11011	33	[ESCAPE]	75	4B	1001011	113	K	123	7B	1110101	173	{
28	1C	11100	34	[FILE SEPARATOR]	76	4C	1001100	114	L	124	7C	1111000	174	
29	1D	11101	35	[GROUP SEPARATOR]	77	4D	1001101	115	M	125	7D	1111001	175	}
30	1E	11110	36	[RECORD SEPARATOR]	78	4E	1001110	116	N	126	7E	1111100	176	~
31	1F	11111	37	[UNIT SEPARATOR]	79	4F	1001111	117	O	127	7F	1111101	177	[DEL]
32	20	100000	40	[SPACE]	80	50	1010000	120	P					
33	21	100001	41	!	81	51	1010001	121	Q					
34	22	100010	42	"	82	52	1010010	122	R					
35	23	100011	43	#	83	53	1010011	123	S					
36	24	100100	44	\$	84	54	1010100	124	T					
37	25	100101	45	%	85	55	1010101	125	U					

- Fonte: <https://upload.wikimedia.org/wikipedia/commons/d/dd/ASCII-Table.svg>

Vamos praticar?

- **Criptografia:** faremos um programa para esconder informações
- Desenvolva uma aplicação em java que troque cada letra de um texto por outra do alfabeto X posições a frente, onde X pode ser escolhido por você (cifra de César). Fonte de exemplo: <https://cryptii.com/pipes/caesar-cipher>

- Exemplo com X=2:

—**“Douglas”** passará a ser **“Fqwincu”**

—**“Alana”** passará a ser **“Cncpc”**



Dúvidas e Questionamentos



INSTITUTO FEDERAL
Mato Grosso do Sul

alana.neo@ifms.edu.br